

ConwayLife Sprint1

Introduction

Goal: Realizzazione del in Java del [GAME OF LIFE DI CONWAY](#) limitatamente al requisito R1

Requirements

- Realizzare una versione in Java del gioco Life di Conway, come gioco zero-player.
Il gioco consiste nell’introdurre una Griglia di Celle il cui stato (cella ‘viva’ o cella ‘morta’) evolve come stabilito dalle regole di ConwayLife.
- L’utente umano deve poter:
 - specificare la configurazione iniziale della griglia del gioco
 - vedere l’evoluzione del gioco in forma opportuna (si veda [Problema della vista del gioco](#))
 - fermare e far ripartire l’evoluzione del gioco
 - pulire (a gioco fermo) la configurazione della griglia del gioco

Requirement analysis

Dalla lettura del requisito R1 si individuano tre entità logiche che riflettono il dominio del problema: Cell, Grid e Life. Procedo con un confronto con il committente.

Che cosa intende il committente per Cell?

Per Cell, dopo aver parlato con il committente, formalizzo il seguente [modello](#).
A livello di struttura Cell è atomica e possiede uno stato (viva o morta).
Avrà quindi bisogno di poter restituire il proprio stato, modificarlo e passare da uno all'altro.

Per definire il comportamento atteso di una cella sviluppo la seguente [TestPlan Unit](#).

Che cosa intende il committente per Grid?

Per Grid, dopo aver parlato con il committente, formalizzo il seguente [modello](#).
A livello di struttura il committente ha un'idea di Grid composta da Cell.
E' un'entità a due dimensioni (una volta fissate non variabili): colonne e righe.
Avrà quindi bisogno di poter restituire le proprie dimesnioni e una sua componente. Aggiungo anche funzioni non primitive per settare lo stato di una cella, ottenerne il valore, contare i vicini di una cella e resettarsi (settare a morte tutte le celle di cui è composta).

Per definire il comportamento atteso di una griglia sviluppo la seguente [TestPlan Unit](#). In particolare voglio assicurarmi che:

- le get delle dimensioni mi restituiscano un natural (≥ 0)
- in tutti i metodi che hanno parametri row e col mi venga restituito NULL se sono al di fuori del range $[0, \text{getDim}-1]$

Che cosa intende il committente per Life?

Il committente non sa cos'è Life, ma mi dice che è la logica del gioco, contiene in sè le regole del gioco.
Formalizzo il seguente [modello](#) che incapsula le regole del gioco.
Strutturalmente, Life è composta da una griglia.
Avrò quindi sicuramente bisogno di un metodo per ottenere la griglia. Inoltre sarà necessario un metodo che vada a farla evolvere secondo le reogle del gioco.

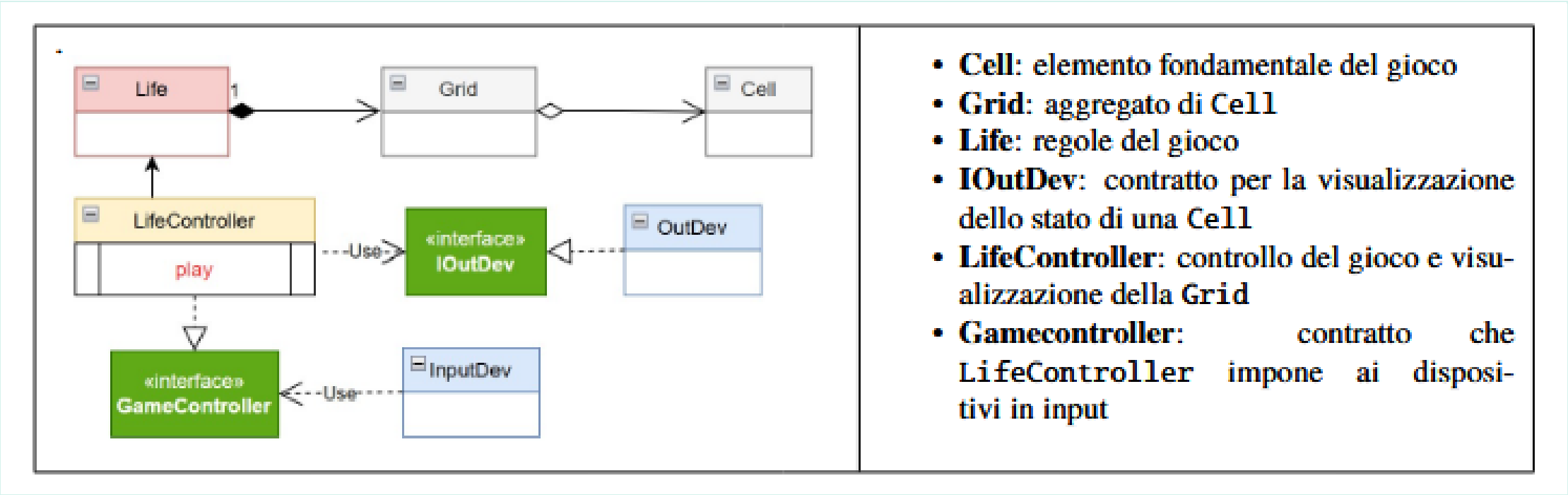
Per definire il comportamento atteso da Life sviluppo la seguente [TestPlan Unit](#). Andrò quindi a inserire delle configurazioni note e a verificare che si evolvano come previsto.

Per descrivere un sistema mi occorrono tre dimensioni: struttura, comportamento e interazione. Fino a questo momento non ho ancora preso in esame la terza componente.
Tecnicamente parlando per interagire con Cell, Grid o Life: chiamo i metodi che offre, trasferimento del controllo (procedure call). Non ne ho parlato perché è implicito che nel mondo Java che si interagisce con procedure call. I miei oggetti sono entità passive. Mi serve quindi un'entità che dia loro la vita.

Inserisco il [LifeController](#): al suo interno ha un thread (play), che infonde vita ai vari elementi, perché gli trasferisce il suo controllo. LifeController mi dice quali sono le interfacce per permettere a qualsiasi dispositivo di visualizzare la griglia. E' componente proattivo (thread play che quando attivato chiama ripetutamente nextGeneration) e reattivo (voglio fare in modo che se c’è un input device che dice start e stop lui parte e si ferma)

Problem analysis

In base a quanto emerso nell'analisi dei requisiti, decido di organizzare il sistema con la seguente architettura logica:



Come indicato in [Il problema della vista del gioco](#), la visualizzazione del gioco non deve essere cablata nel sistema, ma deve essere garantito un disaccoppiamento tra logica e rappresentazione. A questo fine è stata introdotta [IOutDev](#), che fa in modo che sia il dispositivo a dover implementare un contratto definito dall'applicazione: in questo modo è il dispositivo che dipende dall'applicazione e non viceversa (principio di [inversione delle dipendenze](#))

Test plans

Project

Testing

Deployment

Il deployment del sistema avviene grazie all'utilizzo di Gradle come strumento di automazione per la build. Tramite il task jar nel file [build.gradle](#) viene generato il file **conway26Java-1.0.jar**.

Maintenance